

GUÍA DE TRABAJO

N.º 04 Series



Profesor: Guillermo Cid Ampuero

Indicaciones generales

- Resuelva cada ejercicio en un archivo independiente utilizando un formato de nombre claro, por ejemplo: `ejercicio_1.py`, `ejercicio_2.py`, etc.
- Mantenga el estándar de importación de librerías: utilice `import numpy as np` e `import matplotlib.pyplot as plt` cada vez que sean necesarias.
- Cuando el objetivo del problema sea implementar un método numérico desde cero, no utilice funciones integradas de Python (como las de `scipy`) que entreguen la solución de forma directa, a menos que se solicite explícitamente para comparar resultados.
- Priorice la claridad en su código. Utilice comentarios para explicar la lógica de su solución, defina funciones propias cuando sea pertinente y nombre sus variables de forma descriptiva.
- En todos los gráficos generados incluya obligatoriamente: título, nombre de los ejes, unidades de medida, cuadrícula (*grid*) y, cuando exista más de una curva, una leyenda.
- Valide sus resultados: antes de ejecutar un programa, analice el problema e intente anticipar el orden de magnitud o el comportamiento esperado de la solución.

Ejercicio 1 – Aproximación del Seno por Serie de Taylor

La serie de Taylor permite aproximar funciones complejas mediante polinomios. Para la función $f(x) = \sin(x)$ alrededor del punto $a = 0$ (también conocida como Serie de Maclaurin), la aproximación viene dada por:

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = \sum_{n=0}^N \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

Escriba un programa que:

- Importe `numpy`, `matplotlib.pyplot` y la librería `math` (para el factorial).
- Cree un arreglo de valores para x desde -2π hasta 2π con pasos de 0.1 utilizando `np.arange()`. (Hint: Utilice `np.pi`).
- Calcule los valores reales del seno para el arreglo x utilizando `np.sin()`.
- Defina una función `taylor_seno(x, N)` que reciba el arreglo x y la cantidad de términos N , y retorne la aproximación calculando la sumatoria.
- Calcule las aproximaciones para $N = 1$, $N = 3$ y $N = 5$ términos.
- Grafique en una misma figura la función real $\sin(x)$ y las tres aproximaciones obtenidas. Restrinja el eje vertical (eje y) entre -2 y 2 usando `plt.ylim()`.
- Incluya obligatoriamente título, etiquetas de ejes, leyenda y cuadrícula.

Ejercicio 2 – Aproximación Exponencial por Serie de Maclaurin

La serie de Maclaurin para la función exponencial $f(x) = e^x$ es una de las series más fundamentales y está definida como:

$$e^x \approx 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n=0}^N \frac{x^n}{n!}$$

Escriba un programa que:

- Importe las librerías necesarias (`numpy`, `matplotlib.pyplot`, `math`).
- Cree un arreglo de valores para x en el intervalo $[-3, 3]$ con `np.linspace()` utilizando 100 puntos.
- Calcule el valor exacto de la exponencial utilizando `np.exp()`.
- Defina una función `maclaurin_exp(x, N)` que calcule la serie hasta N términos utilizando un ciclo `for`.
- Genere una gráfica que compare la señal original e^x con las aproximaciones para $N = 2$, $N = 4$ y $N = 6$.
- Restrinja el límite del eje y entre 0 y 25 para visualizar correctamente el comportamiento cerca del origen.
- No olvide agregar la leyenda, los nombres de los ejes y la cuadrícula.

Ejercicio 3 – Aproximación del Coseno por Serie de Maclaurin

Al igual que el seno, el coseno puede aproximarse mediante un polinomio considerando sólo potencias pares. La serie de Maclaurin para $f(x) = \cos(x)$ es:

$$\cos(x) \approx 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots = \sum_{n=0}^N \frac{(-1)^n}{(2n)!} x^{2n}$$

Escriba un programa que:

- Defina un punto específico de evaluación $x_0 = \pi/3$ (equivalente a 60°).
- Calcule y guarde en una variable el valor real de $\cos(\pi/3)$.
- Cree una lista vacía llamada `errores`.
- Mediante un ciclo `for`, evalúe la aproximación de Maclaurin variando el número de términos desde $N = 1$ hasta $N = 10$.
- En cada iteración, calcule el error absoluto entre el valor real y la aproximación, y guárdelo en la lista `errores`. (Hint: Utilice `abs()`).
- Grafique el error absoluto en función del número de términos N . Utilice un gráfico de puntos unidos por líneas (`marker='o'`).
- Asigne escalas y etiquetas correspondientes. Imprima en consola el valor del error para $N = 5$.

Ejercicio 4 – Estimación de la Derivada: Diferencia Finita Hacia Adelante

Las diferencias finitas son métodos numéricos que permiten estimar la derivada de una función. La diferencia finita hacia **adelante** se define como:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Considere el siguiente polinomio:

$$f(x) = 0.25x^4 - 0.5x^3 + 0.15x^2 - 0.1x + 2.0$$

Cuya derivada analítica exacta es:

$$f'(x) = x^3 - 1.5x^2 + 0.3x - 0.1$$

Escriba un programa que:

- Defina dos funciones en Python: `f(x)` para el polinomio y `df_real(x)` para su derivada analítica.
- Defina el punto de evaluación $x = 0.5$ y un tamaño de paso $h = 0.25$.
- Calcule el valor de la derivada real evaluada en x .
- Calcule el valor de la derivada aproximada utilizando la fórmula de diferencia finita hacia adelante.
- Calcule el error absoluto entre ambos resultados.
- Imprima en pantalla de forma ordenada: el valor de la derivada real, el valor aproximado y el error absoluto.

Ejercicio 5 – Estimación de la Derivada: Diferencia Finita Hacia Atrás

La estimación mediante diferencia finita hacia **atrás** toma un punto anterior para estimar la pendiente, utilizando la fórmula:

$$f'(x) \approx \frac{f(x) - f(x - h)}{h}$$

Considere un nuevo polinomio dado por:

$$f(x) = -0.3x^4 + 0.2x^3 - 0.6x^2 + 1.5x - 1.0$$

El valor analítico de su derivada es:

$$f'(x) = -1.2x^3 + 0.6x^2 - 1.2x + 1.5$$

Escriba un programa que:

- Defina las funciones de Python `f(x)` y `df_real(x)` correspondientes a este nuevo polinomio.
- Defina el punto de evaluación $x = 1.0$ y un tamaño de paso $h = 0.1$.
- Calcule el valor analítico (real) de la derivada en dicho punto.
- Aproxime la derivada utilizando la fórmula hacia atrás.
- Calcule el error absoluto.
- Muestre por consola los resultados del valor real, el aproximado y el error cometido, acompañados de texto descriptivo.

Ejercicio 6 – Estimación de la Derivada: Diferencia Finita Centrada

Para obtener una mejor precisión sin necesidad de reducir demasiado el paso de integración, suele utilizarse la diferencia finita **centrada**, que promedia las estimaciones hacia adelante y hacia atrás:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Considere el polinomio:

$$f(x) = -0.15x^4 - 0.1x^3 - 0.4x^2 - 0.2x + 1.5$$

Su derivada real está dada por:

$$f'(x) = -0.6x^3 - 0.3x^2 - 0.8x - 0.2$$

Escriba un programa que:

- Defina las rutinas de Python para la función $f(x)$ y su derivada real $df_real(x)$.
- Evalúe en el punto $x = 0.5$ usando un paso grande $h = 0.5$.
- Calcule el valor real de la derivada en $x = 0.5$.
- Calcule la estimación de la derivada utilizando la fórmula centrada.
- Obtenga el error absoluto de esta aproximación.
- Imprima en pantalla los valores calculados de forma clara. (Observe cómo a pesar del gran valor de h , la aproximación centrada ofrece un resultado razonable en comparación con los métodos asimétricos).

Consejos

- ejecute cada programa desde el inicio;
- compruebe que no existan errores de ejecución;
- revise que los resultados sean coherentes;
- verifique las unidades utilizadas;
- compruebe que todos los gráficos tengan título, nombres de los ejes y cuadrícula.